F2: tan(x)

SOEN 6011 — Software Engineering Processes
Deliverable 2

Konan Brandon
Student ID: 40373534

Summer 2026

Contents

1 Introduction

This deliverable reimplements the tangent function from first principles, replaces the textual interface of Deliverable 1 with a graphical one, places the source under public version control, and revises the requirements in light of what the implementation uncovered.

The repository is published at:

https://github.com/Brand0-code/soen6011-tan-calculator

Generative artificial intelligence was used throughout, as required by the project constraints. Each problem below records the tool, the prompt type, the prompt itself structured according to the CASTROFF framework, and an assessment of the output including the points at which it was corrected.

2 Problem 5: Implementation from Scratch

2.1 Interpretation of "from scratch"

The specification permits functions related to input, output, arithmetic and user interface design, and prohibits every other built-in or library function. The implementation therefore does not import `math`. The constant π is a literal, and the sine and cosine are computed from their Maclaurin series. The built-ins that remain are `float` and `int` for conversion, `abs` for arithmetic, `isinstance` for validating the type of the input, and `print` for output.

Meeting this constraint required identifying and implementing the functions subordinate to the tangent, listed in Table 1.

Function	Role
`validate_number`	Rejects non-numeric values, infinities, NaN and out-of-range magnitudes.
`degrees_to_radians`	Multiplies by $\pi/180$, replacing `math.radians`.
`_nearest_integer`	Rounds to the nearest whole number, since `int` truncates towards zero.
`reduce_range`	Subtracts the nearest whole multiple of π.
`sine`, `cosine`	Maclaurin series with an incremental term recurrence.
`compute_tan`	Validates, reduces, checks for an asymptote, and divides.

Table 1: Functions subordinate to $\tan(x)$.

2.2 Series evaluation without factorials

The Maclaurin expansion of the sine is

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$$

Evaluating each term independently would require a factorial and a power, both of which grow rapidly. Dividing one term by its predecessor gives

$$\frac{x^5/5!}{x^3/3!} = \frac{x^2}{4 \cdot 5},$$

so in general each term is obtained from the previous one by multiplying by $-x^2/\big((2k)(2k+1)\big)$. The cosine uses the same recurrence with $-x^2/\big((2k-1)(2k)\big)$. One multiplication and one division per term replace the factorial entirely, nothing overflows, and the loop terminates when a term falls below 10^{-15} rather than after a fixed count.

2.3 Range reduction

The tangent has period π, not 2π, because the sine and cosine both change sign and the signs cancel in the ratio. Subtracting the nearest whole multiple of π therefore leaves the result unchanged while confining the argument to $[-\pi/2, \pi/2]$, where the series converges quickly. Without this step an input such as $x = 1000$ would be unusable. Reduction also collapses every asymptote onto a single check at $\pm\pi/2$.

2.4 Exception handling

Two exception classes are defined, both derived from `TanCalculatorError` so that the interface can catch either with a single handler:

- `UndefinedTangentError` — the angle lies on an asymptote.
- `InvalidInputError` — the value is non-numeric, infinite, NaN, or beyond the validated range.

Messages state what went wrong and what to enter instead, rather than reporting a fault code.

2.5 Graphical user interface

The interface is built with Tkinter and resides in `tan_gui.py`, which performs no arithmetic of its own. It offers an entry field, mutually exclusive radian and degree selection, a Calculate control bound to both a button and the Return key, a Clear control bound to a button and the Escape key, and a result area. Errors appear as plain sentences; a stack trace is never shown. Every control carries a visible text label, focus starts in the entry field, and the outcome is stated in words as well as indicated by colour, so the interface does not depend on colour perception.

The program runs with `python tan_gui.py` from a terminal, with no development environment and no build step.

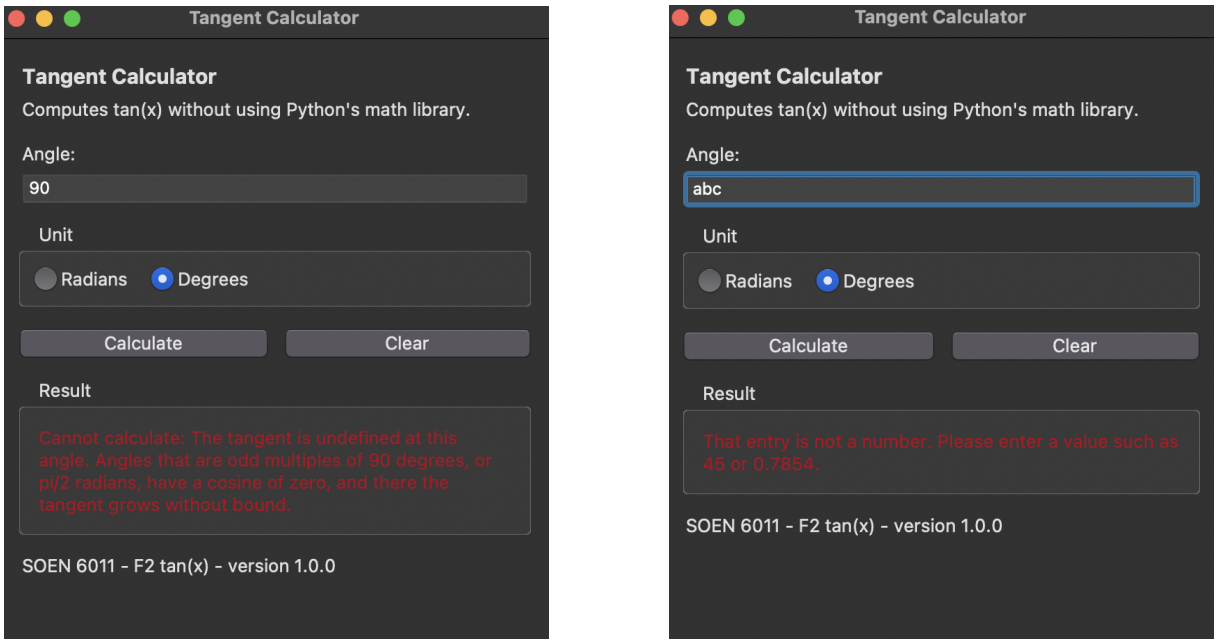

Figure 1: A successful calculation, and an angle on an asymptote reported in plain language.

2.6 Verification

The repository includes `verify_tan.py`, a harness that compares the implementation against Python's `math.tan`. The harness is not part of the implementation: `math` is imported only there. It runs forty-four checks across reference angles, degree conversion, range reduction, near-asymptote behaviour, undefined points, rejected input, and the boundary of the accepted range. All forty-four pass.

2.7 A defect found by testing, and the validated range

Comparing large arguments against the reference revealed a defect that inspection alone would not have caught. For $x = 10^{15}$ the program returned -1.557 where the true value is -1.672: wrong in the first decimal place, with no error raised.

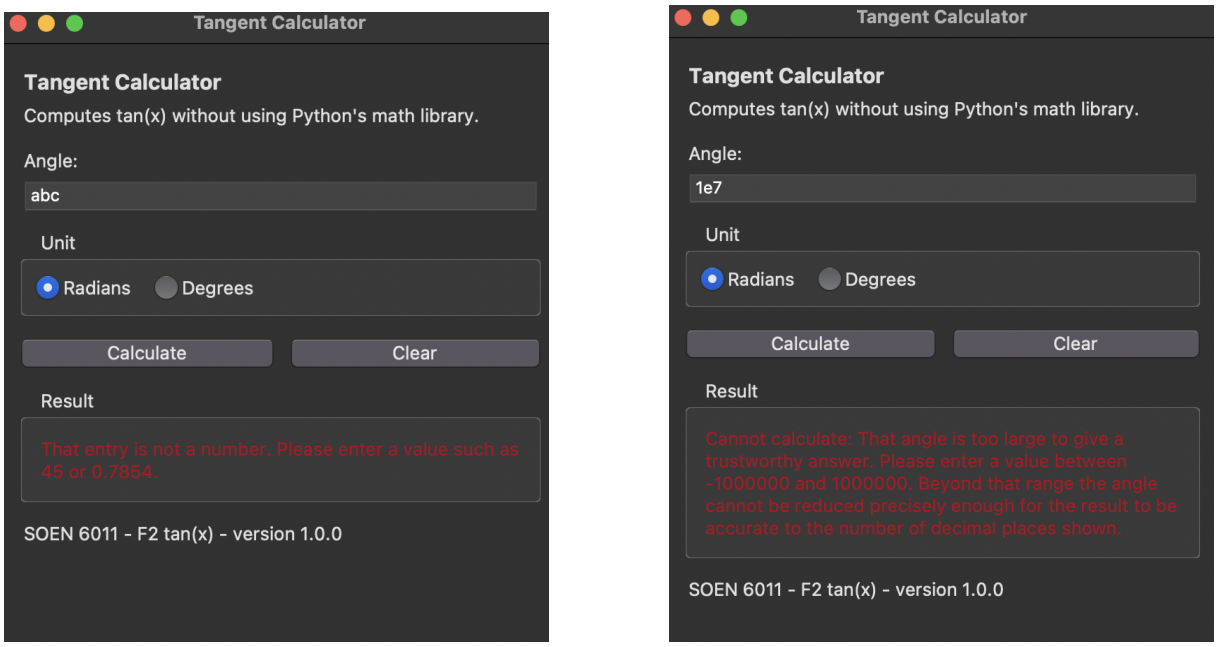

Figure 2: Non-numeric input rejected with guidance, and an angle beyond the validated range refused with an explanation.

Angle (rad)	This program	`math.tan`	Rel. diff.
$\pi/4$	1.0000000000	1.0000000000	1.1×10^{-16}
1.0	1.5574077247	1.5574077247	1.4×10^{-16}
2.0	−2.1850398633	−2.1850398633	2.0×10^{-16}
1000.0	1.4703241557	1.4703241557	3.8×10^{-14}
12345.678	−0.9914971407	−0.9914971407	1.5×10^{-12}
1.57	1255.7655915007	1255.7655915008	3.5×10^{-14}
10^6	−0.3736244539	−0.3736244540	9.4×10^{-11}

Table 2: Selected verification results.

The cause lies in range reduction. Finding the multiple of π to subtract requires computing x/π, and the error in the product $k\pi$ grows with k. Sampling four thousand angles at each order of magnitude gives Table 3.

Results are displayed to ten decimal places. At 10^6 the worst case still holds about seven correct digits; past that it falls below six, and the figures on screen would increasingly be noise presented as precision. The validated range is therefore $\pm 10^6$, and angles beyond it are refused with an explanation rather than answered. Production libraries solve this with Payne–Hanek reduction [5], which is beyond the scope of this deliverable.

Every exact asymptote within the accepted range — 636,620 of them, counting both signs — was tested and correctly raised `UndefinedTangentError`.

2.8 Conformance to style

Both modules and the harness pass Flake8 with a 79-column limit and report no issues, and all three files are rated 10.00 out of 10 by Pylint. Two warnings are suppressed in the source, each with the reason recorded alongside it: `comparison-with-itself` in `tan_math.py`, since comparing a value to itself is the intended way to detect NaN without `math.isnan`, and `too-many-ancestors` in `tan_gui.py`, because `ttk.Frame` already inherits through six classes and any subclass of it exceeds the default limit. Although style tools are a Deliverable 3 requirement, they were applied

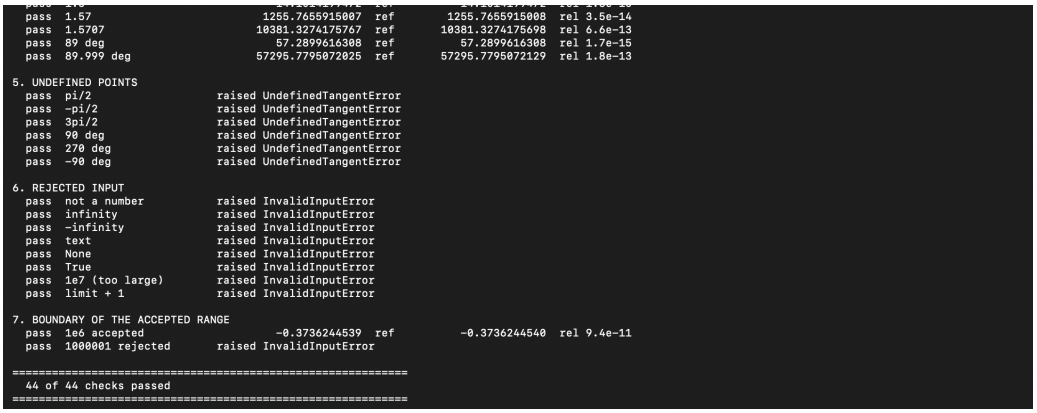

Figure 3: Verification harness: forty-four of forty-four checks passed.

Magnitude	Typical correct digits	Worst observed
10^2	13.6	10.8
10^3	12.7	10.1
10^4	11.7	8.8
10^5	10.7	8.5
10^6	9.7	6.8
10^7	8.7	6.2
10^8	7.7	5.1

Table 3: Correct significant digits against `math.tan`.

here so that later work begins from a clean baseline.

2.9 Use of generative artificial intelligence

Tool: Claude (Anthropic).

Prompt type: role prompting, zero-shot.

Prompt, structured by CASTROFF:

- **Constraints:** no `math` module, no built-in factorial, no built-in trigonometry; own π constant; reduction by the period π; tolerance-based termination; custom exceptions; must run without an IDE.
- **Audience:** a grader checking the from-scratch constraint; the end user is a beginner-level civil engineering student.
- **Structure:** a single module with separate helper functions.
- **Tone:** clean, well-commented, readable by a beginner.
- **Role:** a Python developer implementing strictly to a specification.
- **Output format:** complete runnable code with an explanation of each function.
- **Focus:** correctness against a reference across ordinary, near-asymptote and large inputs.
- **Function:** to satisfy D2/Problem 5.

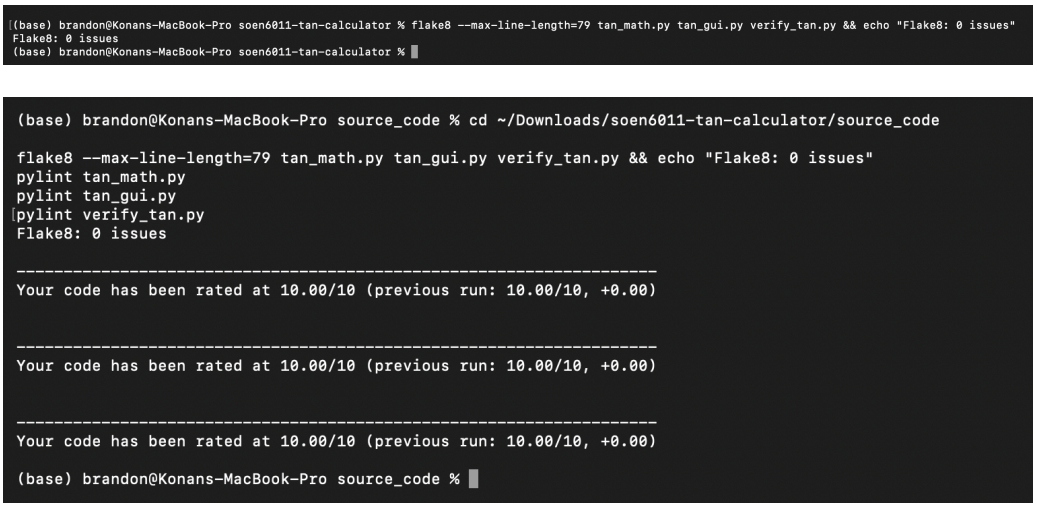

Figure 4: Flake8 reports no issues; Pylint rates all three files 10.00/10.

Explanation of the output. The response supplied a complete module and a rationale for each function. The structure and the term recurrence were adopted. The output was then audited rather than accepted:

1. The loop construct was challenged against the restriction on built-in functions. The response confirmed compliance and enumerated every built-in the module uses, which served as a compliance checklist (Figure 6).

2. Verification against the reference exposed the large-argument defect described above, which the generated code neither prevented nor flagged. Independent review confirmed both the defect and its cause (Figure 7).

3. An initial limit of 10^{10} was proposed. Measuring the surviving digits showed this to be too permissive, and the limit was tightened to 10^{6}. The decision, and the measurement supporting it, are mine rather than the tool's.

A separate prompt produced the interface. That version called `compute_tan` with two arguments, whereas the module defines one, so every calculation would have raised `TypeError`. The fault was found by testing the two modules together rather than by reading either in isolation, and the interface was corrected before use. A final review of the submitted module confirmed compliance and the exhaustive asymptote result (Figure 9).

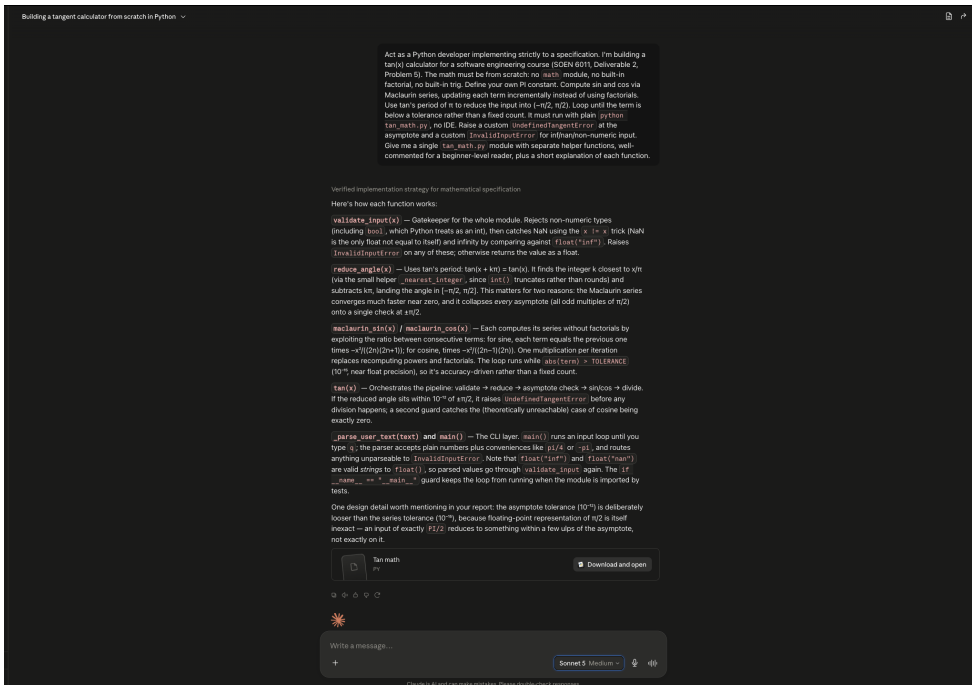

Figure 5: Specification prompt and the generated module.

3 Problem 6: Version Control

3.1 Repository

The source is held in Git and published publicly on GitHub at https://github.com/Brand0-code/soen6011-tan-calculator. The release corresponding to this deliverable is tagged `v1.0.0`, following Semantic Versioning.

3.2 Commit history

Commits were made incrementally as the work progressed rather than as a single deposit. Messages use the imperative mood, and the body explains why the change was made:

```
Add from-scratch tangent module using Maclaurin series

Compute tan(x) as sin(x)/cos(x) without importing math. PI is a
literal constant and each series term is derived from the previous
one, which removes the need for a factorial and prevents overflow.
```

```
Add verification harness comparing output against math.tan

Checks reference angles, degree conversion, range reduction,
behaviour near asymptotes, undefined points, rejected input, and the
boundary of the accepted range. The math import is confined to this
harness and is absent from the implementation.
```

3.3 Repository contents

3.4 Use of generative artificial intelligence

Tool: Claude. **Prompt type:** role prompting with source material supplied for review.

Figure 6: Auditing the generated loops against the built-in restriction.

`tan_math.py`	Calculation only; no interface code.
`tan_gui.py`	Tkinter interface; no calculation code.
`verify_tan.py`	Verification harness; not part of the implementation.
`README.md`	Purpose, usage, verification, accuracy, validated range, error handling, versioning.
`screenshots/`	Interface states and tool output.
`.gitignore`	Excludes interpreter and platform artefacts.

Table 4: Repository layout.

The prompt asked the tool to act as an open-source maintainer reviewing the repository, and supplied the README together with the commit history, asking specifically for anything inaccurate, missing, or inconsistent with the source, and for any point at which the commit message standard slipped.

Explanation of the output. The review identified a defect that had gone unnoticed: the quick-start instructions still directed the reader to run `python tan_gui.py` from the repository root, although the source files had been moved into `source_code/`. The documented command therefore failed on a fresh clone, even though the verification and style-checking sections elsewhere in the same file had been updated correctly. A stale reference to "the two modules" was also found, dating from before the verification harness was written.

The review further noted that the most recent commit message carried no explanatory body, unlike the others, and that the author address recorded in the history was a placeholder rather than a working one. Both were corrected.

This is the same pattern that produced the findings in Problem 5: the fault was visible only when the documented command was actually executed, not when the file was read.

Figure 7: Independent confirmation of the large-argument defect.

4 Problem 7: Updated Requirements

4.1 Rationale for the Revised Identifier Scheme

The requirements in D1/Problem 2 used a single flat sequence (REQ-001 to REQ-008) that mixed functional statements with quality attributes: REQ-001 (compute the tangent) and REQ-007 (numerical precision) belong to different classes of requirement yet shared one numbering space. ISO/IEC/IEEE 29148 distinguishes functional requirements from quality and interface requirements, and expects identifiers to be stable so that a requirement can be traced and versioned independently of its neighbours.

The revised scheme separates requirements into four categories, each with its own sequence:

- **TAN-FR-*nn*** — functional requirements: what the software computes.
- **TAN-UI-*nn*** — user interface requirements: how the user supplies input and receives output.
- **TAN-NFR-*nn*** — non-functional requirements: quality attributes such as precision, performance, usability and accessibility.
- **TAN-CON-*nn*** — constraints: restrictions imposed on the solution rather than on its behaviour.

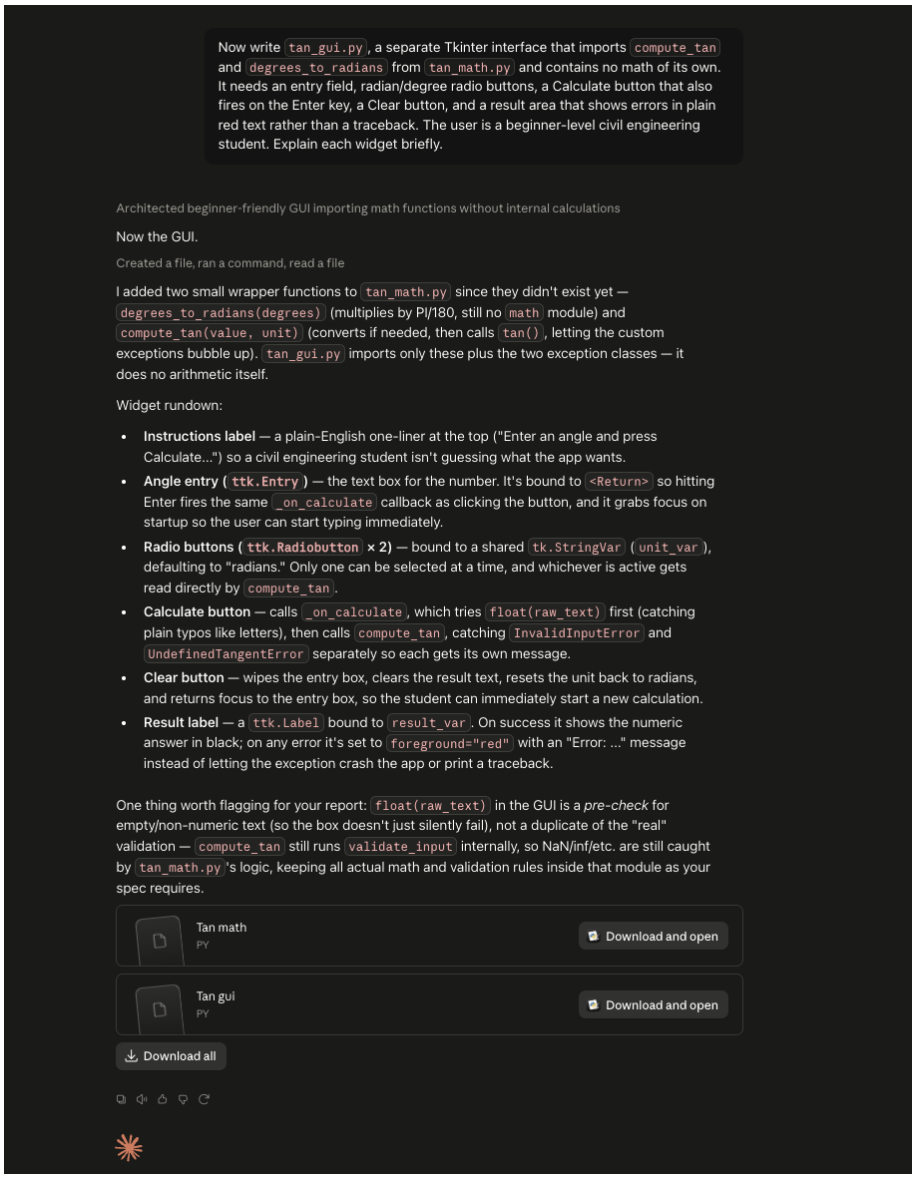

Figure 8: Specification prompt for the Tkinter interface.

Identifiers are not reused. Where a D1 requirement was retained, its origin is recorded so that the earlier document remains traceable.

4.2 Functional Requirements

ID	Requirement	Source / Rationale
TAN-FR-01	The software shall compute $\tan(x)$ for any real-valued angle x within the validated input range.	D1/REQ-001. Priya's primary goal: slope calculations.
TAN-FR-02	The software shall compute $\tan(x)$ using only arithmetic operators and functions defined within the software itself, without calling any library routine that performs part of the calculation.	D2/Problem 5 "from scratch" constraint.

ID	Requirement	Source / Rationale
TAN-FR-03	The software shall evaluate $\tan(x)$ as $\sin(x)/\cos(x)$, with sin and cos obtained from their Maclaurin series.	Design decision recorded in D1/Problem 4.
TAN-FR-04	The software shall reduce the input angle by the nearest whole multiple of π before evaluating the series.	Required for accuracy; the tangent has period π.
TAN-FR-05	The software shall generate each series term from its predecessor by multiplication and division, without computing a factorial.	Avoids overflow and satisfies TAN-FR-02.
TAN-FR-06	The software shall stop summing a series when the magnitude of the current term falls below 10^{-15}, rather than after a fixed number of terms.	Accuracy-driven termination; replaces the fixed term count used in D1.
TAN-FR-07	The software shall report that $\tan(x)$ is undefined, rather than returning a numeric value, when the reduced angle lies within 10^{-10} of an odd multiple of $\pi/2$.	D1/REQ-002, modified: the check now follows range reduction. Priya's frustration with silently wrong results near 90°.
TAN-FR-08	The software shall accept an angle expressed in either radians or degrees, with radians as the default.	D1/REQ-005, modified: the unit is now selected explicitly rather than inferred.
TAN-FR-09	The software shall convert an angle given in degrees to radians internally before computing the tangent.	D1/REQ-006, unchanged. Priya's degrees/radians confusion.
TAN-FR-10	The software shall reject input that is not a finite real number, including infinity and NaN.	D2/Problem 5; not covered in D1.
TAN-FR-11	The software shall reject any angle whose magnitude exceeds 10^6 radians.	Added after verification revealed that range reduction becomes unreliable beyond this magnitude (see D2/Problem 5).
TAN-FR-12	The software shall reject input that cannot be interpreted as a number, including empty input.	Implicit in D1; made explicit following implementation.

4.3 User Interface Requirements

ID	Requirement	Source / Rationale
TAN-UI-01	The software shall provide a graphical user interface built with Tkinter.	D1/REQ-004, modified: supersedes the textual interface required in D1.
TAN-UI-02	The interface shall provide a single-line entry field for the angle, labelled with visible text.	Priya has only brief exposure to programming; the purpose of each control must be evident.

ID	Requirement	Source / Rationale
TAN-UI-03	The interface shall provide a mutually exclusive choice between radians and degrees, with radians selected on start.	Supports TAN-FR-08; prevents the unit being left unspecified.
TAN-UI-04	The interface shall provide a control that starts the calculation, activated either by a button or by the Return key.	Keyboard operation for repeated calculations.
TAN-UI-05	The interface shall provide a control that clears the entry field and the result, activated either by a button or by the Escape key.	Allows the user to recover from a mistake.
TAN-UI-06	The interface shall display the result in a dedicated, consistently positioned area.	Predictable location reduces searching.
TAN-UI-07	The interface shall report every error as a plain sentence within the window, and shall never display a stack trace.	D1/REQ-003, extended to the graphical interface. Priya's goal of understanding *why* a result is undefined, not merely that it is.
TAN-UI-08	The interface shall place keyboard focus in the entry field when the program starts.	The user can type immediately without clicking.

4.4 Non-Functional Requirements

ID	Requirement	Source / Rationale
TAN-NFR-01	All messages presented to the user shall use plain, non-technical language and shall state what to do next.	D1/REQ-003, unchanged. Priya's discomfort with cryptic messages.
TAN-NFR-02	The software shall display results to ten decimal places.	D1/REQ-007, modified: raised from six, since verification showed at least that many digits are reliable within the accepted range.
TAN-NFR-03	Within the validated input range, results shall agree with an independent reference implementation to at least six significant digits.	Added: makes the accuracy claim verifiable rather than assumed.
TAN-NFR-04	The software shall complete a single calculation and display the result within one second.	D1/REQ-008, unchanged. Priya cross-checks manual calculations, so repeated queries must return promptly.
TAN-NFR-05	Every control shall carry a visible text label, shall be reachable from the keyboard, and no outcome shall be conveyed by colour alone.	Added: accessibility, anticipating D3.
TAN-NFR-06	Calculation logic and interface logic shall reside in separate modules, with no arithmetic in the interface module.	Added: allows the calculation to be tested without a window on screen.

4.5 Constraints

Figure 9: Final compliance and accuracy review of the submitted module.

ID	Constraint	Source / Rationale
TAN-CON-01	The software shall execute from a command line using a standard Python interpreter, without requiring an integrated development environment or a build step.	D2/Problem 5.
TAN-CON-02	All error conditions shall be signalled through dedicated exception classes derived from a single base class.	D2/Problem 5; allows one handler to catch every calculator error.
TAN-CON-03	The source code shall conform to PEP 8.	Anticipates D3/Problem 7; already satisfied.

4.6 Change Log Relative to D1

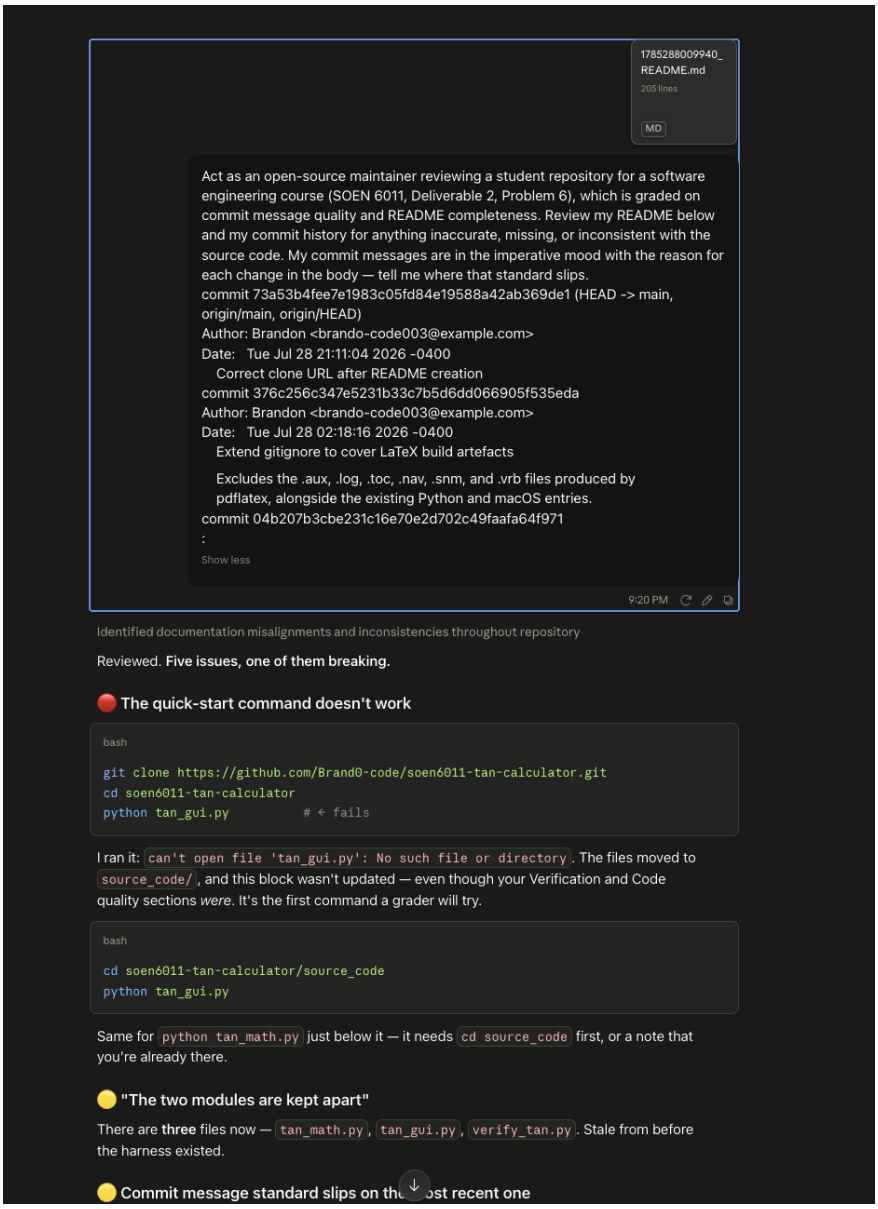

Figure 10: Prompt supplying the README and commit history for review.

D1 ID	Status	Disposition
REQ-001	Unchanged	Becomes TAN-FR-01. Scope now bounded by the validated input range.
REQ-002	Modified	Becomes TAN-FR-07. The tolerance is now a concrete value (10^{-10}) and the check is applied after range reduction, so a single test covers every asymptote.
REQ-003	Unchanged	Split into TAN-NFR-01 (language) and TAN-UI-07 (no stack traces), since one concerns wording and the other the interface.
REQ-004	Modified	Becomes TAN-UI-01. The textual interface is replaced by a Tkinter interface, as required by D2/Problem 5.

D1 ID	Status	Disposition
REQ-005	Modified	Becomes TAN-FR-08. The unit is now selected explicitly rather than assumed, removing the ambiguity noted in D1.
REQ-006	Unchanged	Becomes TAN-FR-09.
REQ-007	Modified	Becomes TAN-NFR-02. Precision raised from six decimal places to ten, supported by measurement.
REQ-008	Unchanged	Becomes TAN-NFR-04.
—	Added	TAN-FR-02, TAN-FR-03, TAN-FR-04, TAN-FR-05, TAN-FR-06, TAN-FR-10, TAN-FR-11, TAN-FR-12, TAN-UI-02 to TAN-UI-06, TAN-UI-08, TAN-NFR-03, TAN-NFR-05, TAN-NFR-06, TAN-CON-01, TAN-CON-02, TAN-CON-03.
—	Removed	None. No D1 requirement was withdrawn; REQ-004 was superseded rather than deleted.

Eight D1 requirements become twenty-nine, of which four are unchanged, four are modified, and twenty-one are new. Most additions arise directly from implementing the function from scratch: the prohibition on library routines (TAN-FR-02) implies decisions about series evaluation, range reduction and termination that were left unstated in D1, and each of those decisions carries its own verifiable requirement.

4.7 Assumptions

- "Real-valued" excludes complex numbers, which remain out of scope.
- The validated input range ($\pm 10^6$ radians) is a property of this implementation, not of the tangent function. It follows from double-precision arithmetic and the range reduction method chosen.
- The reference implementation named in TAN-NFR-03 is Python's `math.tan`. It is used only for verification and is never called by the software itself.
- The one-second limit in TAN-NFR-04 assumes ordinary desktop hardware and excludes interpreter start-up.
- Accessibility in TAN-NFR-05 is addressed at the level of labelling, keyboard operation and non-reliance on colour. Screen-reader conformance is not claimed.

4.8 Use of generative artificial intelligence

Tool: Claude. **Prompt type:** chain-of-thought — the model was asked to reason about the category, atomicity and verifiability of each requirement before restating it.

The prompt supplied the eight requirements from Deliverable 1 together with the feedback that the identifier scheme mixed functional and non-functional requirements, and asked for a categorised scheme conforming to ISO/IEC/IEEE 29148 with every requirement traced to the persona or to a constraint arising from Problem 5. The output was revised in two respects: REQ-003 was split, because plain language is a quality attribute while the absence of a stack trace is an interface behaviour, and the wording of the accuracy requirement was tightened so that it names a verifiable reference rather than asserting precision in the abstract.

Figure 11: Findings of the review, including the broken quick-start command.

5 Conclusion

The tangent is now computed entirely from first principles, presented through a graphical interface, held under public version control, and described by a requirements set that reflects what the implementation actually does.

The most instructive part of the work was not writing the series but measuring it. Comparing the implementation against an independent reference exposed a defect that reading the code would not have revealed, and choosing where to draw the validated range required measurement rather than judgement alone. The same applies to the integration fault between the two modules: each file was correct in isolation and wrong together.

References

[1] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering*, International Organization for Standardization, 2018.

[2] G. van Rossum, B. Warsaw, and A. Coghlan, "PEP 8 — Style Guide for Python Code," Python Software Foundation, 2001. [Online]. Available: https://peps.python.org/

pep-0008/

[3] D. Goodger and G. van Rossum, "PEP 257 — Docstring Conventions," Python Software Foundation, 2001. [Online]. Available: https://peps.python.org/pep-0257/

[4] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. New York, NY, USA: Dover, 1965.

[5] J.-M. Muller, *Elementary Functions: Algorithms and Implementation*, 3rd ed. Boston, MA, USA: Birkhäuser, 2016.

[6] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, Mar. 1991.

[7] Python Software Foundation, "tkinter — Python interface to Tcl/Tk." [Online]. Available: https://docs.python.org/3/library/tkinter.html

[8] T. Preston-Werner, "Semantic Versioning 2.0.0." [Online]. Available: https://semver.org/

[9] Anthropic, "Claude," large language model, 2026. [Online]. Available: https://claude.ai/